

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6
«Работа с БД в СУБД MongoDB»
по дисциплине «Проектирование и реализация баз данных»**

Обучающийся: Тихая Алиса Сергеевна

Факультет: прикладной информатики

Группа: К3241

Направление подготовки: 09.03.03 – Мобильные и сетевые технологии

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург 2026

Цель работы: овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

2 CRUD-ОПЕРАЦИИ В СУБД MONGODB. ВСТАВКА ДАННЫХ. ВЫБОРКА ДАННЫХ

2.1 ВСТАВКА ДОКУМЕНТОВ В КОЛЛЕКЦИЮ

Практическое задание 2.1.1:

1. *Создайте базу данных learn.*
2. *Заполните коллекцию единорогов unicorns:*

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
```

```
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
```

```
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
```

```
db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
```

```
db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});
```

```
db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
```

```
db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
```

```
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
```

```
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
```

```
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
```

```
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

3. *Используя второй способ, вставьте в коллекцию единорогов документ:*
{name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}

4. *Проверьте содержимое коллекции с помощью метода find.*

```
learn> use learn
already on db learn
learn> db.unicorns.insertMany([
  {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63},
  {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
  {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
  {name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
  {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80},
  {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
  {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
  {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
  {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
  {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
  {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a18706b432cb844f5abc120'),
    '1': ObjectId('6a18706b432cb844f5abc121'),
    '2': ObjectId('6a18706b432cb844f5abc122'),
    '3': ObjectId('6a18706b432cb844f5abc123'),
    '4': ObjectId('6a18706b432cb844f5abc124'),
    '5': ObjectId('6a18706b432cb844f5abc125'),
    '6': ObjectId('6a18706b432cb844f5abc126'),
    '7': ObjectId('6a18706b432cb844f5abc127'),
    '8': ObjectId('6a18706b432cb844f5abc128'),
    '9': ObjectId('6a18706b432cb844f5abc129'),
    '10': ObjectId('6a18706b432cb844f5abc12a')
  }
}
learn> document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
db.unicorns.insertOne(document)
{
  acknowledged: true,
  insertedId: ObjectId('6a187071432cb844f5abc12b')
}
```

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a18706b432cb844f5abc120'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc121'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc122'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc123'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc124'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
```

2.2 ВЫБОРКА ДАННЫХ ИЗ БД

Практическое задание 2.2.1:

1. Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.
2. Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций `findOne` и `limit`.

```
learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a18706b432cb844f5abc121'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc125'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc128'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
```

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a187071432cb844f5abc12b'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc120'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc126'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'})
[
  {
    _id: ObjectId('6a18706b432cb844f5abc121'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc124'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc12a'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a18706b432cb844f5abc121'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a18706b432cb844f5abc121'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Практическое задание 2.2.2:

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
learn> db.unicorns.find(
|   {gender: 'm'},
|   {loves: 0, gender: 0}
| )
|
| [
|   {
|     _id: ObjectId('6a18706b432cb844f5abc120'),
|     name: 'Horny',
|     weight: 600,
|     vampires: 63
|   },
|   {
|     _id: ObjectId('6a18706b432cb844f5abc122'),
|     name: 'Unicrom',
|     weight: 984,
|     vampires: 182
|   },
|   {
|     _id: ObjectId('6a18706b432cb844f5abc123'),
|     name: 'Rooooooodles',
|     weight: 575,
|     vampires: 99
|   },
|   {
|     _id: ObjectId('6a18706b432cb844f5abc126'),
|     name: 'Kenny',
|     weight: 690,
|     vampires: 39
|   },
|   {
|     _id: ObjectId('6a18706b432cb844f5abc127'),
|     name: 'Raleigh',
|     weight: 421,
|     vampires: 2
|   },
|   {
|     _id: ObjectId('6a18706b432cb844f5abc129'),
|     name: 'Pilot',
|     weight: 650,
|     vampires: 54
|   },
|   {
|     _id: ObjectId('6a187071432cb844f5abc12b'),
|     name: 'Dunx',
|     weight: 704,
|     vampires: 165
|   }
| ]
learn>
```

Практическое задание 2.2.3:

Вывести список единорогов в обратном порядке добавления.

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a187071432cb844f5abc12b'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc12a'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc129'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc128'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc127'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a18706b432cb844f5abc126'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
]
```

```
}  
{  
  _id: ObjectId('6a18706b432cb844f5abc125'),  
  name: 'Ayna',  
  loves: [ 'strawberry', 'lemon' ],  
  weight: 733,  
  gender: 'f',  
  vampires: 40  
},  
{  
  _id: ObjectId('6a18706b432cb844f5abc124'),  
  name: 'Solnara',  
  loves: [ 'apple', 'carrot', 'chocolate' ],  
  weight: 550,  
  gender: 'f',  
  vampires: 80  
},  
{  
  _id: ObjectId('6a18706b432cb844f5abc123'),  
  name: 'Rooooooodles',  
  loves: [ 'apple' ],  
  weight: 575,  
  gender: 'm',  
  vampires: 99  
},  
{  
  _id: ObjectId('6a18706b432cb844f5abc122'),  
  name: 'Unicrom',  
  loves: [ 'energon', 'redbull' ],  
  weight: 984,  
  gender: 'm',  
  vampires: 182  
},  
{  
  _id: ObjectId('6a18706b432cb844f5abc121'),  
  name: 'Aurora',  
  loves: [ 'carrot', 'grape' ],  
  weight: 450,  
  gender: 'f',  
  vampires: 43  
},  
{  
  _id: ObjectId('6a18706b432cb844f5abc120'),  
  name: 'Horny',  
  loves: [ 'carrot', 'papaya' ],  
  weight: 600,  
  gender: 'm',  
  vampires: 63  
}
```


Практическое задание 2.1.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
learn> db.unicorns.find(
|   {},
|   {_id: 0, name: 1, loves: {$slice: 1}}
| )
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
```

2.3 ЛОГИЧЕСКИЕ ОПЕРАТОРЫ

Практическое задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
learn> db.unicorns.find(
|   {gender: 'f', weight: {$gte: 500, $lte: 700}},
|   {_id: 0}
| )
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Практическое задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find(
|   {gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}},
|   {_id: 0}
| )
[
|   {
|     name: 'Kenny',
|     loves: [ 'grape', 'lemon' ],
|     weight: 690,
|     gender: 'm',
|     vampires: 39
|   }
| ]
```

Практическое задание 2.3.3:

Найти всех единорогов, не имеющих ключ vampires.

```
learn> db.unicorns.find(
|   {vampires: {$exists: false}}
| )
[
|   {
|     _id: ObjectId('6a18706b432cb844f5abc12a'),
|     name: 'Nimue',
|     loves: [ 'grape', 'carrot' ],
|     weight: 540,
|     gender: 'f'
|   }
| ]
```

Практическое задание 2.3.4:

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find(
|   {gender: 'm'},
|   {_id: 0, name: 1, loves: {$slice: 1}}
| ).sort({name: 1})
[
|   { name: 'Dunx', loves: [ 'grape' ] },
|   { name: 'Horny', loves: [ 'carrot' ] },
|   { name: 'Kenny', loves: [ 'grape' ] },
|   { name: 'Pilot', loves: [ 'apple' ] },
|   { name: 'Raleigh', loves: [ 'apple' ] },
|   { name: 'Roooooodles', loves: [ 'apple' ] },
|   { name: 'Unicrom', loves: [ 'energon' ] }
| ]
```

3 ЗАПРОСЫ К БАЗЕ ДАННЫХ MONGODB. ВЫБОРКА ДАННЫХ. ВЛОЖЕННЫЕ ОБЪЕКТЫ. ИСПОЛЬЗОВАНИЕ КУРСОРОВ. АГРЕГИРОВАННЫЕ ЗАПРОСЫ. ИЗМЕНЕНИЕ ДАННЫХ

3.1 ЗАПРОС К ВЛОЖЕННЫМ ОБЪЕКТАМ

Практическое задание 3.1.1:

1. *Создайте коллекцию towns, включающую следующие документы:*

```
{name: "Punxsutawney ",  
populatiuon: 6200,  
last_sensus: ISODate("2008-01-31"),  
famous_for: [""],  
mayor: {  
  name: "Jim Wehrle"  
}}
```

```
{name: "New York",  
populatiuon: 22200000,  
last_sensus: ISODate("2009-07-31"),  
famous_for: ["status of liberty", "food"],  
mayor: {  
  name: "Michael Bloomberg",  
  party: "I"}}
```

```
{name: "Portland",  
populatiuon: 528000,  
last_sensus: ISODate("2009-07-20"),  
famous_for: ["beer", "food"],  
mayor: {  
  name: "Sam Adams",  
  party: "D"}}
```

2. *Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.*
3. *Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.*

```

learn> db.towns.insertMany([
  {
    name: "Punxsutawney",
    populatiuon: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: [""],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {
    name: "New York",
    populatiuon: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"
    }
  },
  {
    name: "Portland",
    populatiuon: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"
    }
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a1874c8432cb844f5abc12c'),
    '1': ObjectId('6a1874c8432cb844f5abc12d'),
    '2': ObjectId('6a1874c8432cb844f5abc12e')
  }
}

```

```

learn> db.towns.find()
[
  {
    _id: ObjectId('6a1874c8432cb844f5abc12c'),
    name: 'Punxsutawney',
    populatiuon: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ '' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6a1874c8432cb844f5abc12d'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a1874c8432cb844f5abc12e'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]

```

```
learn> db.towns.find(
|   {"mayor.party": "I"},
|   {_id: 0, name: 1, mayor: 1}
| )
|
| [ { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } } ]
|
learn>
```

```
learn> db.towns.find(
|   {"mayor.party": {$exists: false}},
|   {_id: 0, name: 1, mayor: 1}
| )
|
| [ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
|
learn>
```

Практическое задание 3.1.2:

1. Сформировать функцию для вывода списка самцов единорогов.
2. Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.
3. Вывести результат, используя *forEach*.
4. Содержание коллекции единорогов *unicorns*:

```
db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
```

```
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
```

```
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
```

```
db.unicorns.insert({name: 'Rooodoodles', 44), loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
```

```
db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
```

```
db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
```

```
db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
```

```
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
```

```
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
```

```
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
```

```
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

```
db.unicorns.insert({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165});
```

```
learn> fn = function() {
|   return this.gender == "m";
| }
| [Function: fn]
learn>
```

```
learn> var cursor = db.unicorns.find({gender: "m"}).sort({name: 1}).limit(2)

learn> cursor.forEach(function(obj){
|   print(obj.name);
| })
Dunx
Horny
```

3.2 АГРЕГИРОВАННЫЕ ЗАПРОСЫ

Практическое задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.find({
|   gender: "f",
|   weight: {$gte: 500, $lte: 600}
| }).count()
2
learn>
```

Практическое задание 3.2.2:

Вывести список предпочтений.

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn>
```

Практическое задание 3.2.3:

Посчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.aggregate([
|   {
|     $group: {
|       _id: "$gender",
|       count: {$sum: 1}
|     }
|   }
| ])
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
learn>
```

3.3 РЕДАКТИРОВАНИЕ ДАННЫХ

Практическое задание 3.3.1:

1. *Выполнить команду:*
> db.unicorns.save({name: 'Barney', loves: ['grape'],
weight: 340, gender: 'm'})
2. *Проверить содержимое коллекции unicorns.*

```
learn> db.unicorns.save({
|   name: 'Barney',
|   loves: ['grape'],
|   weight: 340,
|   gender: 'm'
| })
TypeError: db.unicorns.save is not a function
```

Save нет в новой версии, использовала insertOne

```
learn> db.unicorns.insertOne({
|   name: 'Barney',
|   loves: ['grape'],
|   weight: 340,
|   gender: 'm'
| })
{
  acknowledged: true,
  insertedId: ObjectId('6a1877de432cb844f5abc12f')
}
learn> db.unicorns.find({name: "Barney"})
[
  {
    _id: ObjectId('6a1877de432cb844f5abc12f'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
```

```
learn> db.unicorns.find({name: "Barney"})
[
  {
    _id: ObjectId('6a1877de432cb844f5abc12f'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
learn>
```

Практическое задание 3.3.2:

1. Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.
2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.updateOne(
|   {name: "Ayna"},
|   {$set: {weight: 800, vampires: 51}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>
learn> db.unicorns.find({name: "Ayna"})
[
  {
    _id: ObjectId('6a18706b432cb844f5abc125'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
learn>
```

Практическое задание 3.3.3:

1. Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.
2. Проверить содержимое коллекции unicorns.

```
learn>
| db.unicorns.updateOne(
|   {name: "Raleigh"},
|   {$push: {loves: "redbull"}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: "Raleigh"})
[
  {
    _id: ObjectId('6a18706b432cb844f5abc127'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn>
```


Практическое задание 3.3.4:

1. Всем самцам единорогов увеличить количество убитых вампиров на 5.
2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.updateMany(  
  {gender: "m"},  
  {$inc: {vampires: 5}}  
)  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 8,  
  modifiedCount: 8,  
  upsertedCount: 0  
}
```

```
learn> db.unicorns.find({gender: "m"})  
[  
  {  
    _id: ObjectId('6a18706b432cb844f5abc120'),  
    name: 'Horny',  
    loves: [ 'carrot', 'papaya' ],  
    weight: 600,  
    gender: 'm',  
    vampires: 68  
  },  
  {  
    _id: ObjectId('6a18706b432cb844f5abc122'),  
    name: 'Unicrom',  
    loves: [ 'energon', 'redbull' ],  
    weight: 984,  
    gender: 'm',  
    vampires: 187  
  },  
  {  
    _id: ObjectId('6a18706b432cb844f5abc123'),  
    name: 'Roooooodles',  
    loves: [ 'apple' ],  
    weight: 575,  
    gender: 'm',  
    vampires: 104  
  },  
  {  
    _id: ObjectId('6a18706b432cb844f5abc126'),  
    name: 'Kenny',  
    loves: [ 'grape', 'lemon' ],  
    weight: 690,  
    gender: 'm',  
    vampires: 44  
  },  
  {  
    _id: ObjectId('6a18706b432cb844f5abc127'),  
    name: 'Raleigh',  
    loves: [ 'apple', 'sugar', 'redbull' ],  
    weight: 421,  
    gender: 'm',  
    vampires: 7  
  },  
  {  
    _id: ObjectId('6a18706b432cb844f5abc129'),  
    name: 'Pilot',  
    loves: [ 'apple', 'watermelon' ],  
    weight: 650,  
    gender: 'm',  
    vampires: 59  
  },  
  {  
    _id: ObjectId('6a187071432cb844f5abc12b'),  
    name: 'Dunx',  
    loves: [ 'grape', 'watermelon' ],  
    weight: 704,  
    gender: 'm',  
    vampires: 170  
  },  
  {  
    _id: ObjectId('6a1877de432cb844f5abc12f'),  
    name: 'Barny',  
    loves: [ 'grape' ],  
    weight: 340,  
    gender: 'm',  
    vampires: 5  
  }  
]
```

Практическое задание 3.3.5:

1. Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.
2. Проверить содержимое коллекции towns.

```
learn> db.towns.updateOne(
|   {name: "Portland"},
|   {$unset: {"mayor.party": 1}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({name: "Portland"})
[
  {
    _id: ObjectId('6a1874c8432cb844f5abc12e'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn>
```

Практическое задание 3.3.6:

1. Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.
2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.updateOne(
|   {name: "Pilot"},
|   {$push: {loves: "chocolate"}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: "Pilot"})
[
  {
    _id: ObjectId('6a18706b432cb844f5abc129'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn>
```

Практическое задание 3.3.7:

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.
2. Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.updateOne(
|   {name: "Aurora"},
|   {$addToSet: {loves: {$each: ["sugar", "lemon"]}}}
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
learn> db.unicorns.find({name: "Aurora"})
[
|   {
|     _id: ObjectId('6a18706b432cb844f5abc121'),
|     name: 'Aurora',
|     loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
|     weight: 450,
|     gender: 'f',
|     vampires: 43
|   }
| ]
learn>
```

3.4 УДАЛЕНИЕ ДАННЫХ ИЗ КОЛЛЕКЦИИ

Практическое задание 3.4.1:

1. Создайте коллекцию towns, включающую следующие документы:

```
{name: "Punxsutawney ",
popujatiuon: 6200,
last_sensus: ISODate("2008-01-31"),
famous_for: ["phil the groundhog"],
mayor: {
  name: "Jim Wehrle"
}}

{name: "New York",
popujatiuon: 22200000,
last_sensus: ISODate("2009-07-31"),
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
  party: "I"}}

{name: "Portland",
popujatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams",
  party: "D"}}
```

2. Удалите документы с беспартийными мэрами.
3. Проверьте содержание коллекции.
4. Очистите коллекцию.
5. Просмотрите список доступных коллекций.

```
learn> db.towns.deleteMany({
|   "mayor.party": {$exists: false}
| })
{ acknowledged: true, deletedCount: 1 }
learn>
```

```
learn> db.towns.find()
[
  {
    _id: ObjectId('6a187aeb432cb844f5abc131'),
    name: 'New York',
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a187aeb432cb844f5abc132'),
    name: 'Portland',
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

```
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
learn>
```

```
learn> show collections
towns
unicorns
learn>
```

4 ССЫЛКИ И РАБОТА С ИНДЕКСАМИ В БАЗЕ ДАННЫХ MONGODB

4.1 ССЫЛКИ В БД

Практическое задание 4.1.1:

1. Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.
2. Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.
3. Проверьте содержание коллекции единорогов.
4. Содержание коллекции единорогов

unicorns:

```
learn> db.habitats.insertMany([
  {
    _id: "forest",
    full_name: "Magic Forest",
    description: "Лесная зона обитания единорогов"
  },
  {
    _id: "mountain",
    full_name: "Crystal Mountain",
    description: "Горная зона обитания единорогов"
  },
  {
    _id: "lake",
    full_name: "Silver Lake",
    description: "Озерная зона обитания единорогов"
  }
])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'mountain', '2': 'lake' }
}
learn>
learn> db.unicorns.updateOne(
  {name: "Aurora"},
  {$set: {habitat: {$ref: "habitats", $id: "forest"}}}
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  {name: "Unicrom"},
  {$set: {habitat: {$ref: "habitats", $id: "mountain"}}}
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  {name: "Leia"},
  {$set: {habitat: {$ref: "habitats", $id: "lake"}}}
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>
```

```
learn> db.unicorns.find(
|   {habitat: {$exists: true}},
|   {_id: 0, name: 1, habitat: 1}
| )
|
| [
|   { name: 'Aurora', habitat: DBRef('habitats', 'forest') },
|   { name: 'Unicrom', habitat: DBRef('habitats', 'mountain') },
|   { name: 'Leia', habitat: DBRef('habitats', 'lake') }
| ]
learn>
```

4.2 НАСТРОЙКА ИНДЕКСОВ

Практическое задание 4.2.1:

1. Проверьте, можно ли задать для коллекции *unicorns* индекс для ключа *name* с флагом *unique*.

```
learn> db.unicorns.createIndex(
|   {name: 1},
|   {unique: true}
| )
name_1
learn> db.unicorns.getIndexes()
[
|   { v: 2, key: { _id: 1 }, name: '_id_' },
|   { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
| ]
learn> db.unicorns.insertOne({
|   name: "Aurora",
|   loves: ["carrot"],
|   weight: 450,
|   gender: "f"
| })
MongoServerError: E11000 duplicate key error collection: learn.unicorns index: name_1 dup key: { name: "Aurora" }
```

4.3 УПРАВЛЕНИЕ ИНДЕКСАМИ

Практическое задание 4.3.1:

1. Получите информацию о всех индексах коллекции *unicorns*.
2. Удалите все индексы, кроме индекса для идентификатора.
3. Попробуйте удалить индекс для идентификатора.

```
learn> db.unicorns.getIndexes()
[
|   { v: 2, key: { _id: 1 }, name: '_id_' },
|   { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
| ]
learn>
learn> db.unicorns.dropIndexes()
{
|   nIndexesWas: 2,
|   msg: 'non-_id indexes dropped for collection',
|   ok: 1
| }
learn>
```

```
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn>
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```

4.4 ПЛАН ЗАПРОСА

Практическое задание 4.4.1:

1. Создайте объемную коллекцию *numbers*, задействовав курсор:
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
2. Выберите последних четыре документа.
3. Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра *executionTimeMillis*)
4. Создайте индекс для ключа *value*.
5. Получите информацию о всех индексах коллекции *numbers*.
6. Выполните запрос 2.
7. Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?
8. Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

```
learn> for(i = 0; i < 100000; i++){
|   db.numbers.insertOne({value: i})
| }
{
  acknowledged: true,
  insertedId: ObjectId('6a1880ef432cb844f5b05513')
}
learn>
```

```
learn> db.numbers.countDocuments()
100000
learn>
```

```
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a1880ef432cb844f5b05513'), value: 99999 },
  { _id: ObjectId('6a1880ef432cb844f5b05512'), value: 99998 },
  { _id: ObjectId('6a1880ef432cb844f5b05511'), value: 99997 },
  { _id: ObjectId('6a1880ef432cb844f5b05510'), value: 99996 }
]
learn>
```

```
learn> db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
executionTimeMillis = 87
```

```

executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 87,
  totalKeysExamined: 0,
  totalDocsExamined: 100000,
  executionStages: {
    isCached: false,
    stage: 'SORT',
  }
}
learn> db.numbers.createIndex({value: 1})
value_1
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn>
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a1880ef432cb844f5b05513'), value: 99999 },
  { _id: ObjectId('6a1880ef432cb844f5b05512'), value: 99998 },
  { _id: ObjectId('6a1880ef432cb844f5b05511'), value: 99997 },
  { _id: ObjectId('6a1880ef432cb844f5b05510'), value: 99996 }
]
learn>

```

executionTimeMillis = 0

```

executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 0,
  totalKeysExamined: 4,
  totalDocsExamined: 4,
  executionStages: {
    isCached: false,
    stage: 'LIMIT',
  }
}

```

```

stage: 'IXSCAN',
nReturned: 4,
executionTimeMillisEstimate: 0,
works: 4,
advanced: 4,
needTime: 0,
needYield: 0,
saveState: 0,
restoreState: 0,
isEOF: 0,
nss: 'learn.numbers',
keyPattern: { value: 1 },
indexName: 'value_1',

```


Вывод

В ходе выполнения лабораторной работы были изучены основные возможности СУБД MongoDB. Были выполнены CRUD-операции: добавление, поиск, изменение и удаление документов. Были изучены методы сортировки, фильтрации, ограничения выборки, работа с логическими операторами, массивами и вложенными объектами. Также были освоены курсоры, агрегатные запросы и операции обновления данных. Были изучены способы создания и использования индексов, а также проанализирован план выполнения запросов с помощью метода `explain()`.